B3: Curves and Sound

CS1101S: Programming Methodology

Low Kok Lim

September 1, 2023

Outline

- **Curves**
- **Sound**

Mastery Checks

- **Oral tests**
- Conducted in 3-way meetings:
  - **2 students**, 1 Avenger, mostly of the same Studio
- **Anytime** during the semester
- **Topics** for **Mastery Check 1** (3% of total course grade)
  - *Scope*
  - *Higher-order functions*
  - *Substitution model and iterative/recursive processes*
- **Pass/fail** for each topic
  - **Repetition allowed** per topic

Mastery Checks: The Process

- Form pairs
  - Your Avenger will help you find a partner from a different Studio if necessary
- Make appointment with Avenger
- For each topic, you have 10 minutes to convince the Avenger that <u>both</u> of you have mastered the topic
- Use proper English and terminology; illustrate your argument with examples
- Be ready for using examples given by the Avenger
- Be ready to answer questions on the topic posed by Avenger
- Avenger will pass you on a topic only if s/he is convinced you’ve mastered it
- The pair can have repeat attempt(s) with the Avenger on unsuccessful topic(s)
- Other staff may check as well for repeated attempts
- Try finish MC1 before Recess Week (great prep for Midterm!)

Outline

- **Curves**

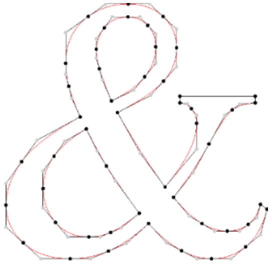
- **Sound**

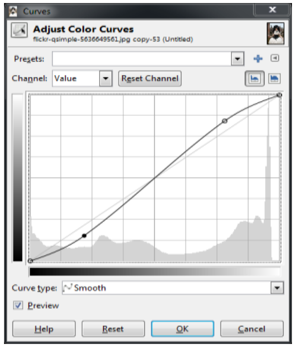
Path, Missions & Quests on Curves

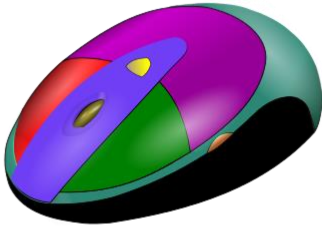
- **Path: “Curves”**
- **Mission: “Curve Introduction”**
- **Mission: “Curve Manipulation”**
- **Mission: “Beyond the First Dimension”**
- **Quest: “Cardioid Arrest”**
- **Quest: “Curvaceous Wizardry”**
- **Contest: “The Choreographer”**

Applications of Curves

- **Drawing** “vector-based”, “infinite resolution” smooth curves
- **Font** design, representation and rendering
- Smooth **animation paths** (for objects, light, camera)
- Designing **smooth functions**
  - E.g. for image color & tone adjustment
- **3D model design**, representation and rendering
  - For engineering/industrial designs, movie production and game development
- **Data fitting**
- etc.







Representation of Curves — **Explicit Form**

- **Curves in 2D**
  - The value of the **dependent variable** is given in terms of the **independent variable**
    - $y = f(x)$
    - **Example:** $y = mx + h$ (straight line)
  - Some curves cannot be expressed in explicit form
    - **Examples:** vertical straight line, circle

- **Curves in 3D**
  - Requires **two equations**
    - $y = f(x)$
    - $z = g(x)$

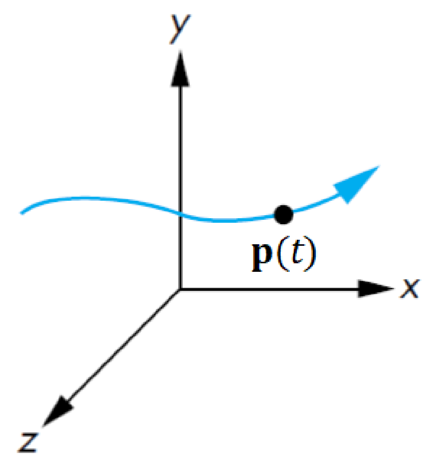
Representation of Curves — **Implicit Form**

- **Curves in 2D**
  - $f(x, y) = 0$
  - Examples:
    - $ax + by + c = 0$ (straight line)
    - $x^2 + y^2 - r^2 = 0$ (circle)
- **Curves in 3D**
  - Can be represented as the **intersection of two surfaces**
    - $f(x, y, z) = 0$
    - $g(x, y, z) = 0$
- A **drawback**: Difficult to obtain points on the curves
  - Because the equations are just membership tests

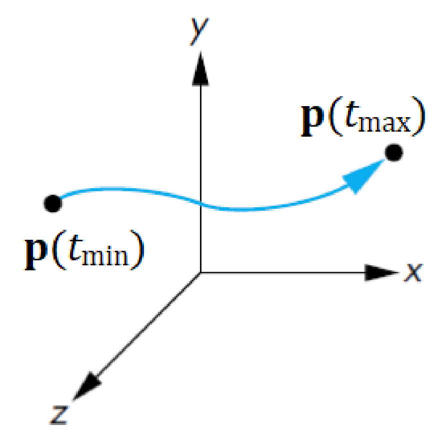
Representation of Curves — **Parametric Form**

- **Curves in 2D and 3D**
  - Each **spatial variable** for points on the curve is expressed in terms of an independent variable t, the **parameter**

$$\mathbf{p}(t) = \begin{bmatrix} x(t) \\ y(t) \end{bmatrix} \qquad \mathbf{p}(t) = \begin{bmatrix} x(t) \\ y(t) \\ z(t) \end{bmatrix}$$



- A **curve segment** is defined for $t_{\min} \leq t \leq t_{\max}$
  - Often, $0 \leq t \leq 1$



Curves in Source

- Supported by the [`curve module`]
- Uses **parametric representation**
- Parameter ***t*** is within the **unit interval [0, 1]**
  - If ***C*** is a Curve, its
    - **starting** point is ***C***(0)
    - **ending** point is ***C***(1)

Specifications of Curves in Source

Curve

Curve := number → Point

A Curve is a **function** that takes a **number** as argument and returns a **Point**. The Number argument must be within the interval [0, 1].

Point

`make_point` : (number, number) → Point
`x_of`, `y_of` : Point → number

```
x_of(make_point(n, m)) = n
y_of(make_point(n, m)) = m
```

Defining Curves

- **Examples:**

```
function unit_circle(t) {
    return make_point(math_cos(2 * math_PI * t),
                      math_sin(2 * math_PI * t));
}

function unit_line_at(y) {
    return t => make_point(t, y);
}

const unit_line = unit_line_at(0);
```

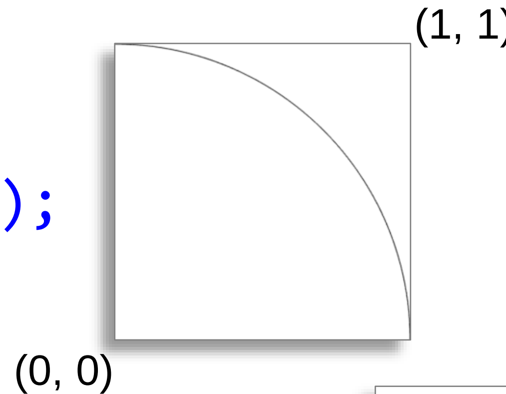
Show in Playground

Drawing Curves

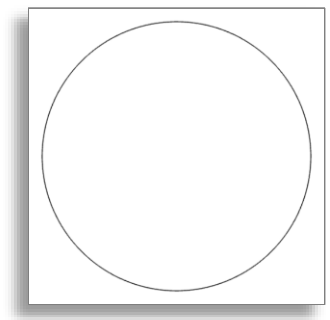
- **Examples:**

Show in Playground

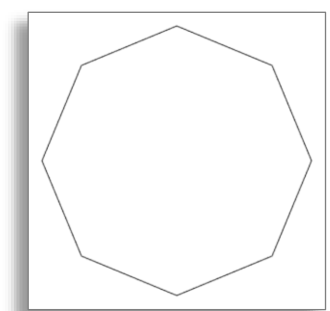
```
draw_connected(200)(unit_circle);
```




```
draw_connected_full_view_proportional(200)
    (unit_circle);
```

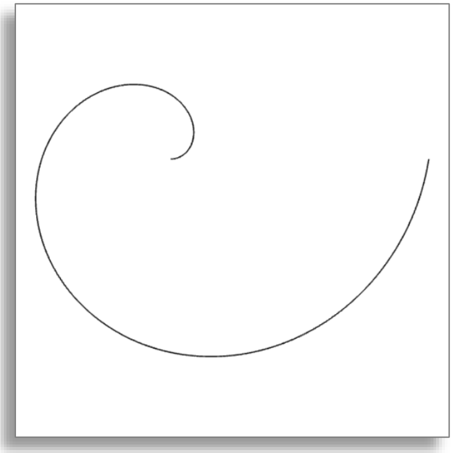


```
draw_connected_full_view_proportional(8)
    (unit_circle);
```



Example: Spiral Curves

- **Wanted:** Write a function `spiral_one` that represents a **spiral curve**
  - Uses `unit_circle`
  - One revolution only



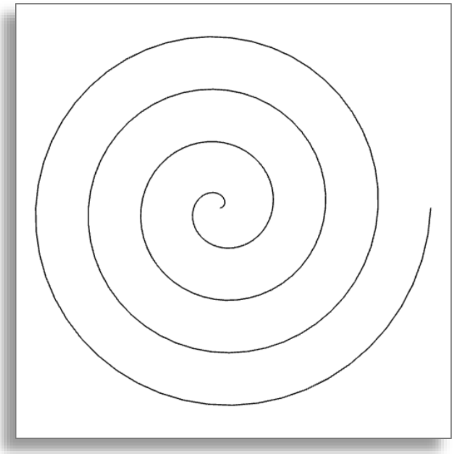
- **Solution:**

```
function spiral_one(t) {
    const p = unit_circle(t);
    return make_point(t * x_of(p), t * y_of(p));
}

draw_connected_full_view_proportional(200)(spiral_one);
```

Show in Playground

Example: Spiral Curves

- **Wanted:** Write a function `spiral` that returns a **spiral curve**
  - Uses `unit_circle`
  - Number of **revolutions** is a parameter

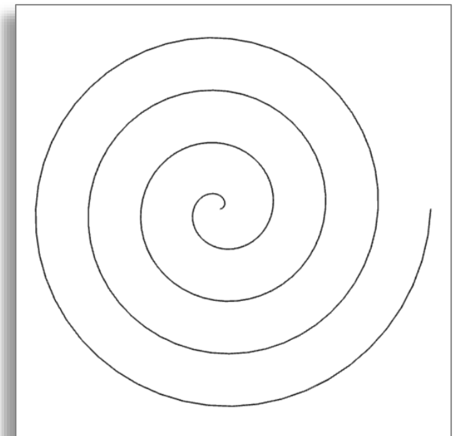


- **Attempt #1:**

```
function spiral(rev, t) {
    const p = unit_circle((t * rev) % 1);
    return make_point(t * x_of(p), t * y_of(p));
}

draw_connected_full_view_proportional(200)(spiral);
```

[Show in Playground]()

Example: Spiral Curves



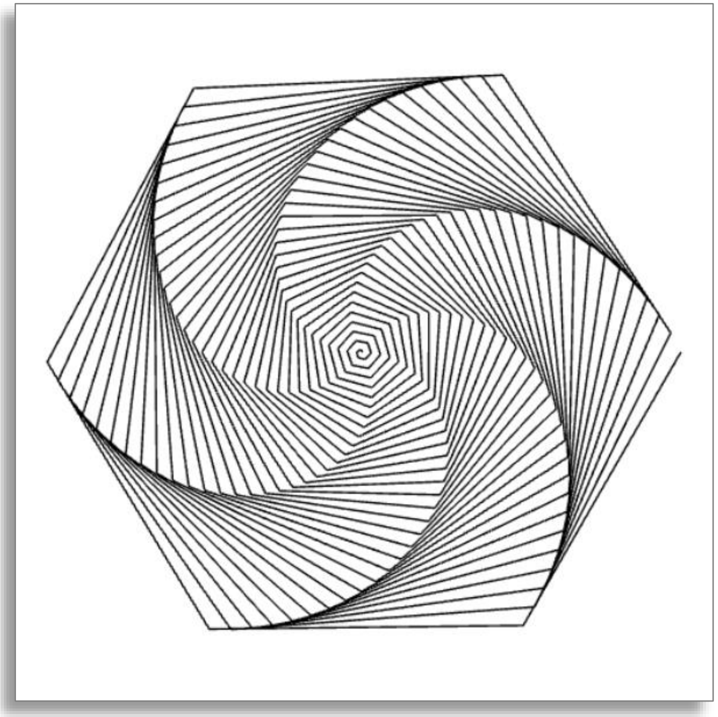
- **Wanted:** Write a function `spiral` that returns a **spiral curve**
  - Uses `unit_circle`
  - Number of **revolutions** is a parameter
- **Attempt #2:**

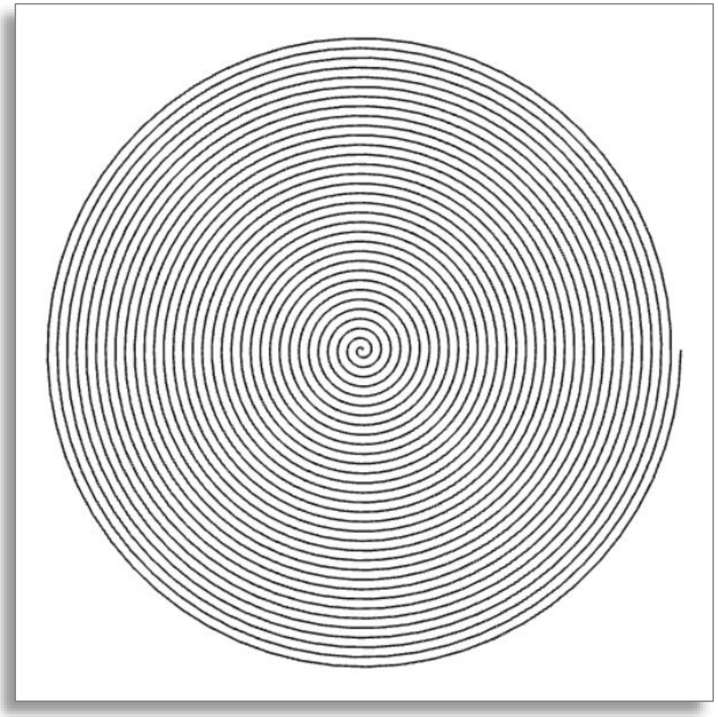
```
function spiral(rev) {
    return t => {
        const p = unit_circle((t * rev) % 1);
        return make_point(t * x_of(p), t * y_of(p));
    };
}
draw_connected_full_view_proportional(200)(spiral(4));
```

Show in Playground

Example: Spiral Curves

```
draw_connected_full_view_proportional(200)(spiral(33));

draw_connected_full_view_proportional(2000)(spiral(33));
```



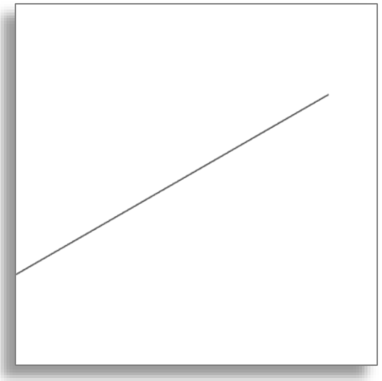


[Show in Playground]

Transformations on Curves

- **Example:**

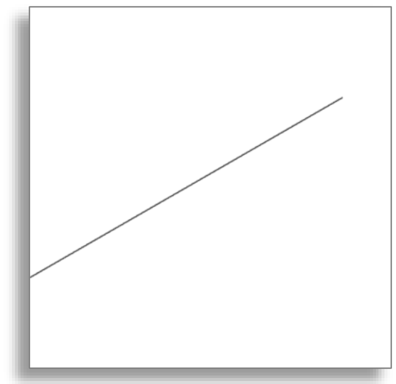
```
const rot_line =
    rotate_around_origin(0, 0, math_PI / 6)(unit_line);

const shifted_rot_line =
    translate(0, 0.25, 0)(rot_line);

draw_connected(200)(shifted_rot_line);
```



[Show in Playground]

Transformations on Curves

- **Example (alternative):**

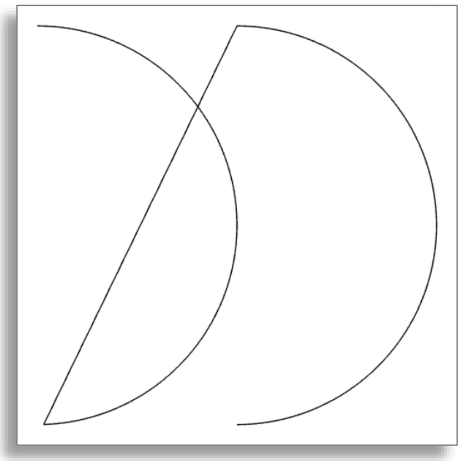


```
function compose(f, g) {
    return x => f(g(x));
}

const shift_rot =
    compose(translate(0, 0.25, 0),
            rotate_around_origin(0, 0, math_PI / 6));

const shifted_rot_line = shift_rot(unit_line);

draw_connected(200)(shifted_rot_line);
```

[Show in Playground]

Connecting Curves

- **Example:**



```
function connect_rigidly(curve1, curve2) {
    return t => t < 1/2
                ? curve1(2 * t)
                : curve2(2 * t - 1);
}

const result_curve =
    connect_rigidly(arc, translate(1, 0, 0)(arc));

draw_connected_full_view_proportional(200)
    (result_curve);
```

[Show in Playground]

Colored Curves

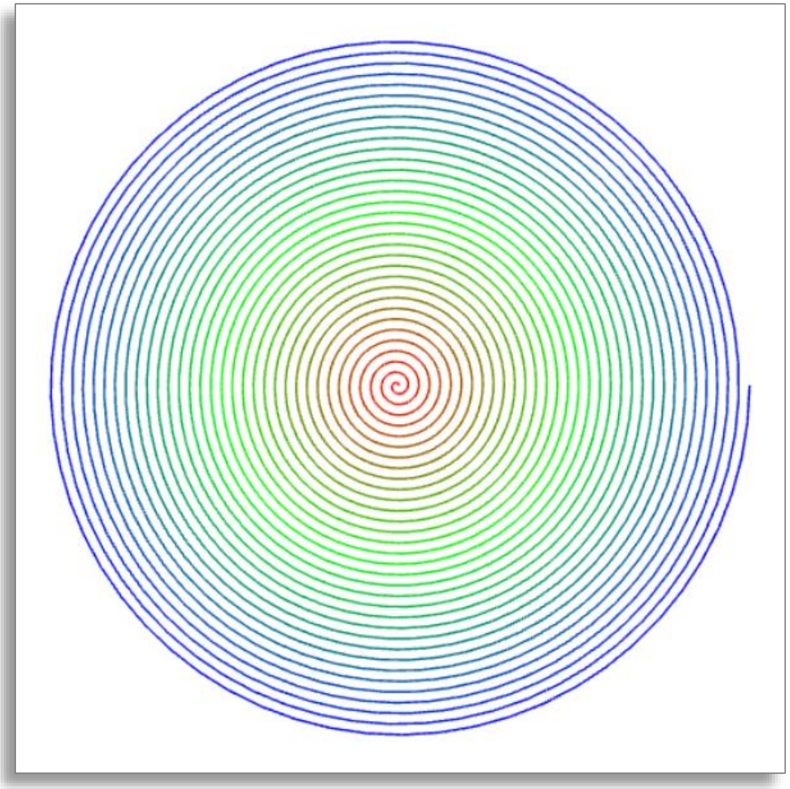
- **Example:**

```
function colorful_spiral(rev) {
    return t => {
        const p = unit_circle((t * rev) % 1);
        const R = math_max(0, 1 - 2 * t) * 255;
        const G = (1 - math_abs(1 - 2 * t)) * 255;
        const B = math_max(0, 2 * t - 1) * 255;
        return make_color_point(t * x_of(p), t * y_of(p),
                                R, G, B);
    };
}
draw_connected_full_view_proportional(2000)
    (colorful_spiral(33));
```

Show in Playground

Colored Curves

```
draw_connected_full_view_proportional(2000)
    (colorful_spiral(33));
```



Show in Playground

3D Curves

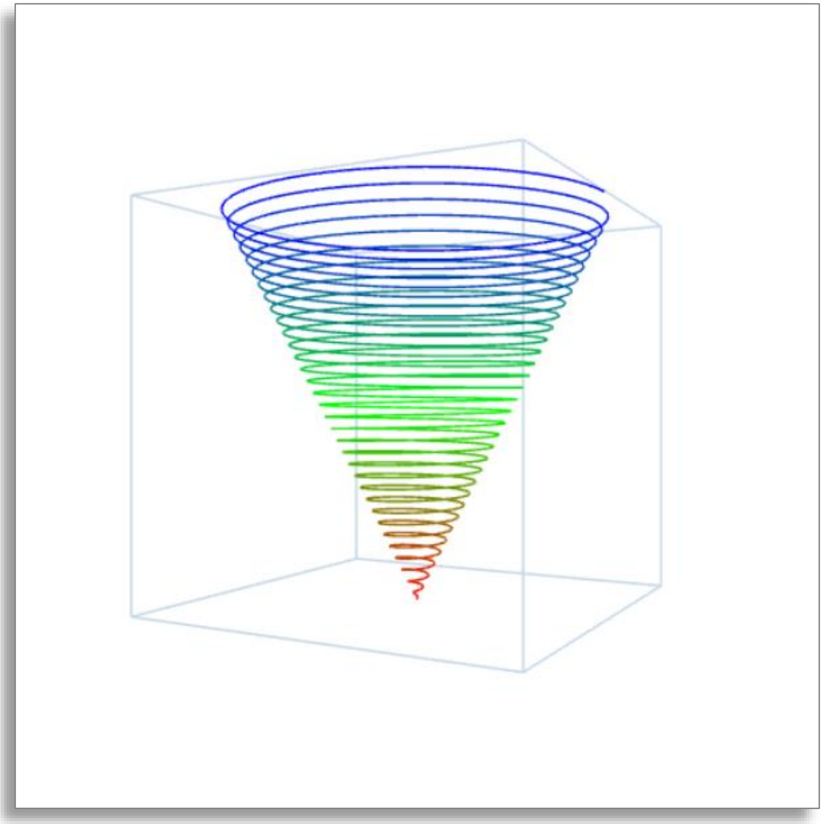
- **Example:**

```
function colorful_3D_spiral(rev) {
    return t => {
        const p = unit_circle((t * rev) % 1);
        const R = math_max(0, 1 - 2 * t) * 255;
        const G = (1 - math_abs(1 - 2 * t)) * 255;
        const B = math_max(0, 2 * t - 1) * 255;
        return make_3D_color_point(
            t * x_of(p), t * y_of(p), 2 * t, R, G, B);
    };
}
draw_3D_connected_full_view_proportional(2000)
    (colorful_3D_spiral(33));
```

Show in Playground

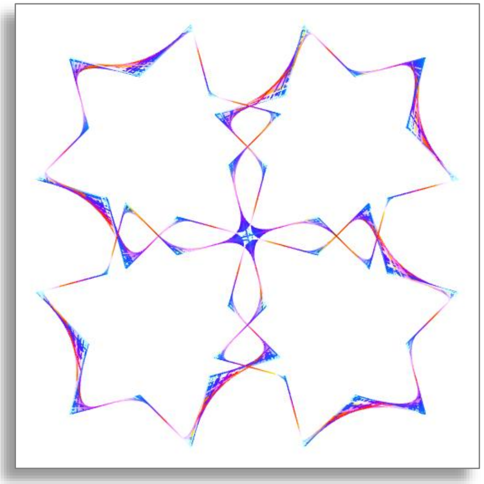
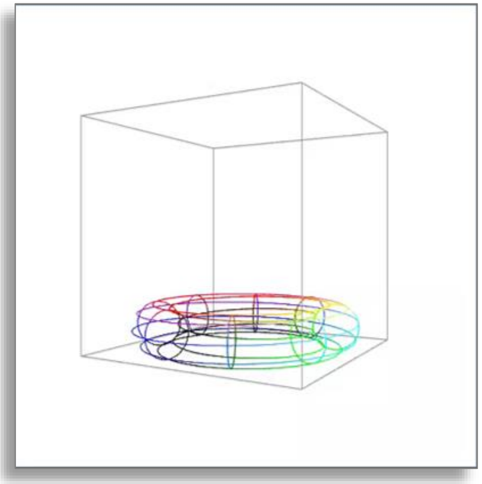
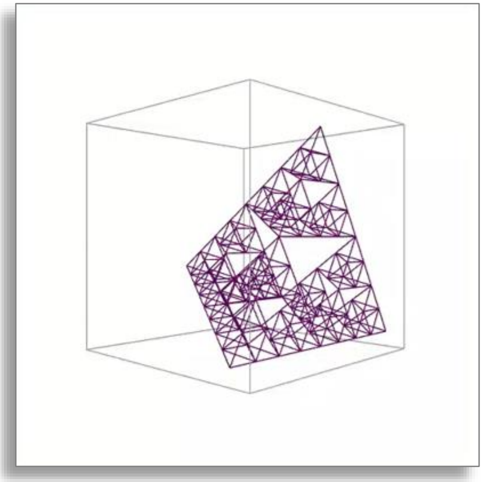
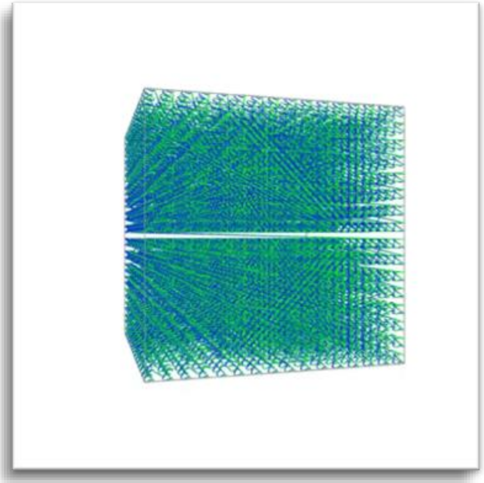
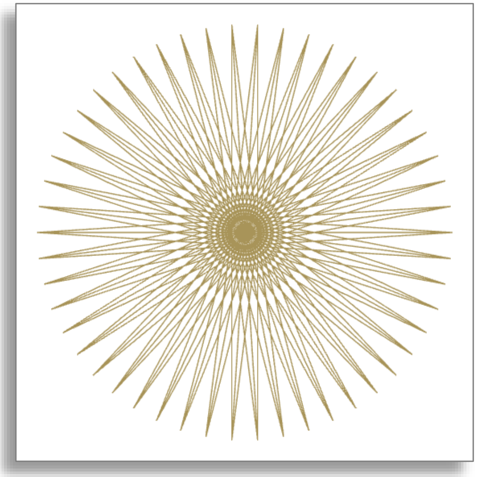
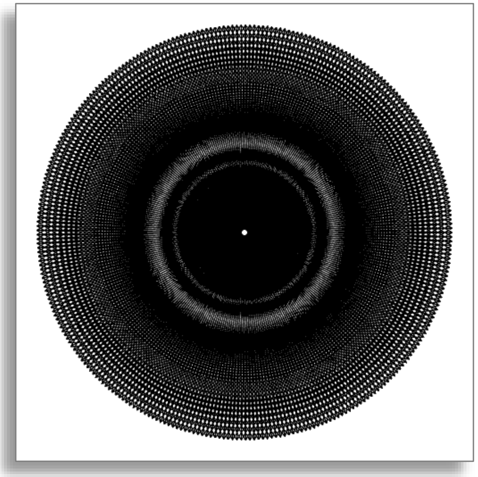
3D Curves

```
draw_3D_connected_full_view_proportional(2000)
    (colorful_3D_spiral(33));
```



Show in Playground

Curves from Past Contests

Summary

- **Functions** can provide **abstractions** for
  - compound operations
  - “**objects**” or **data**

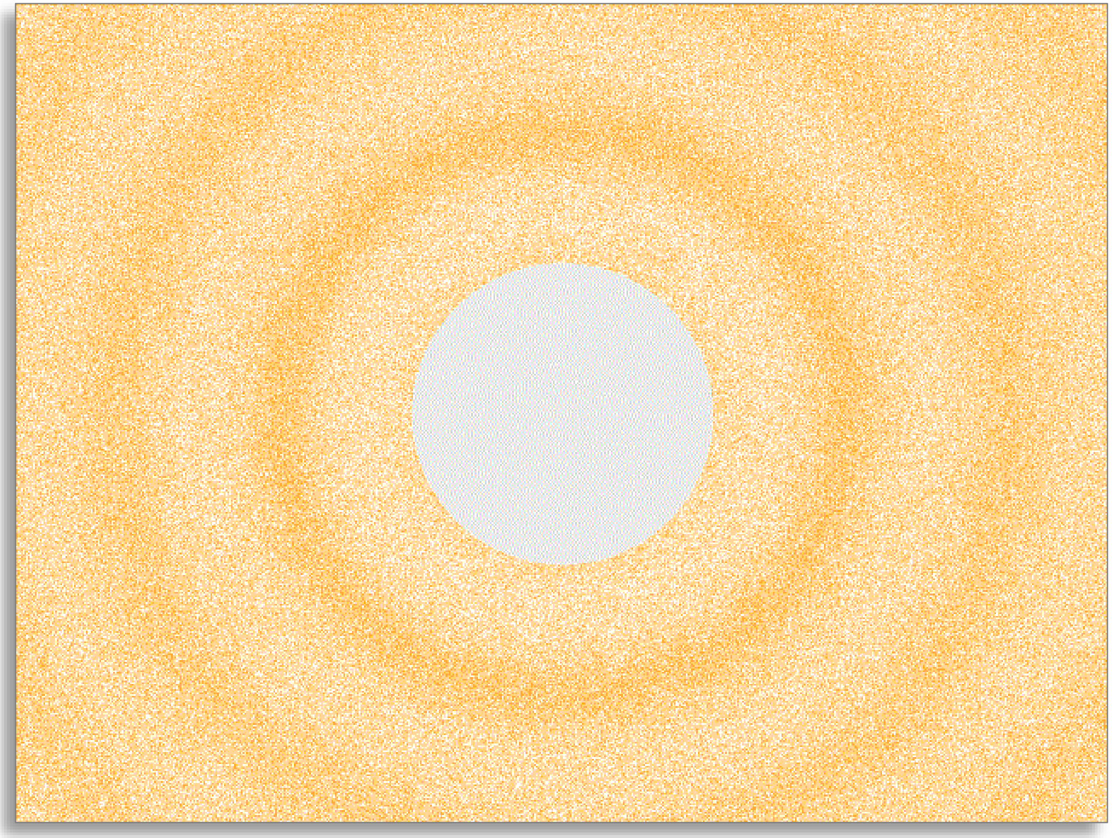
Outline

- Curves
- **Sound**

Missions & Quests on Sound

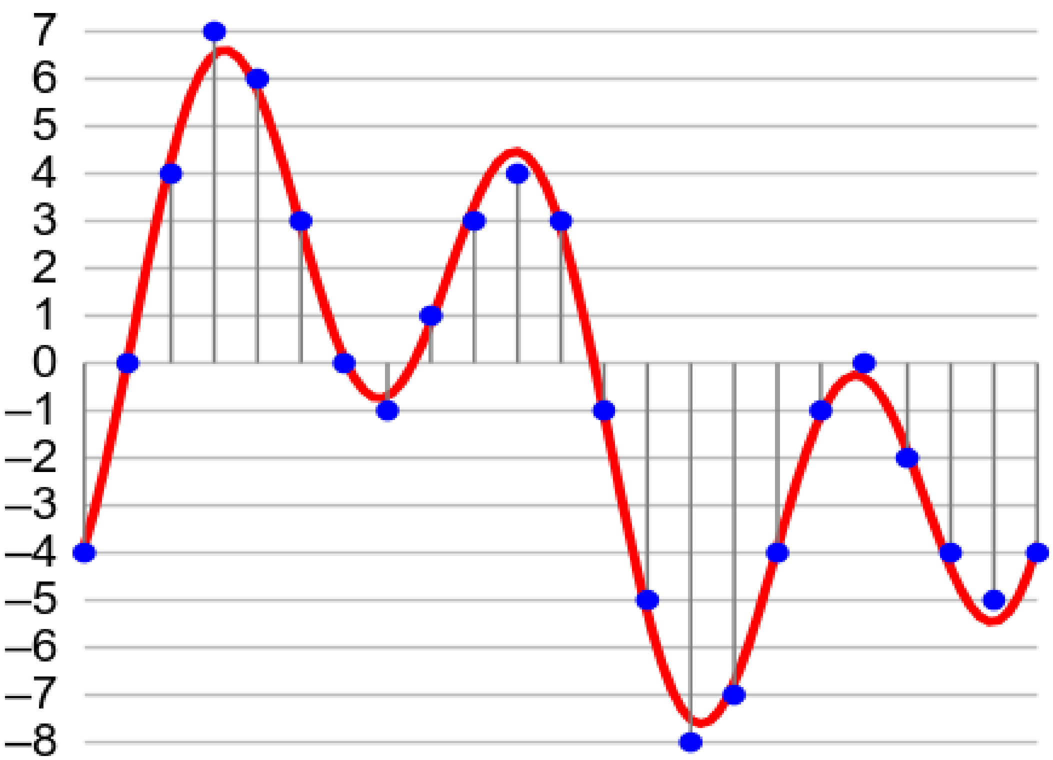
- **Mission: “Premorseal Communications”**
- **Mission: “POTS and Pans”**
- **Mission: “Musical Diversions”**
- **Quest: “Echoes of the Past”**
- **Quest: “The Magical Tone Matrix”**
- **Contest: “Game of Tones”**

What is Sound?



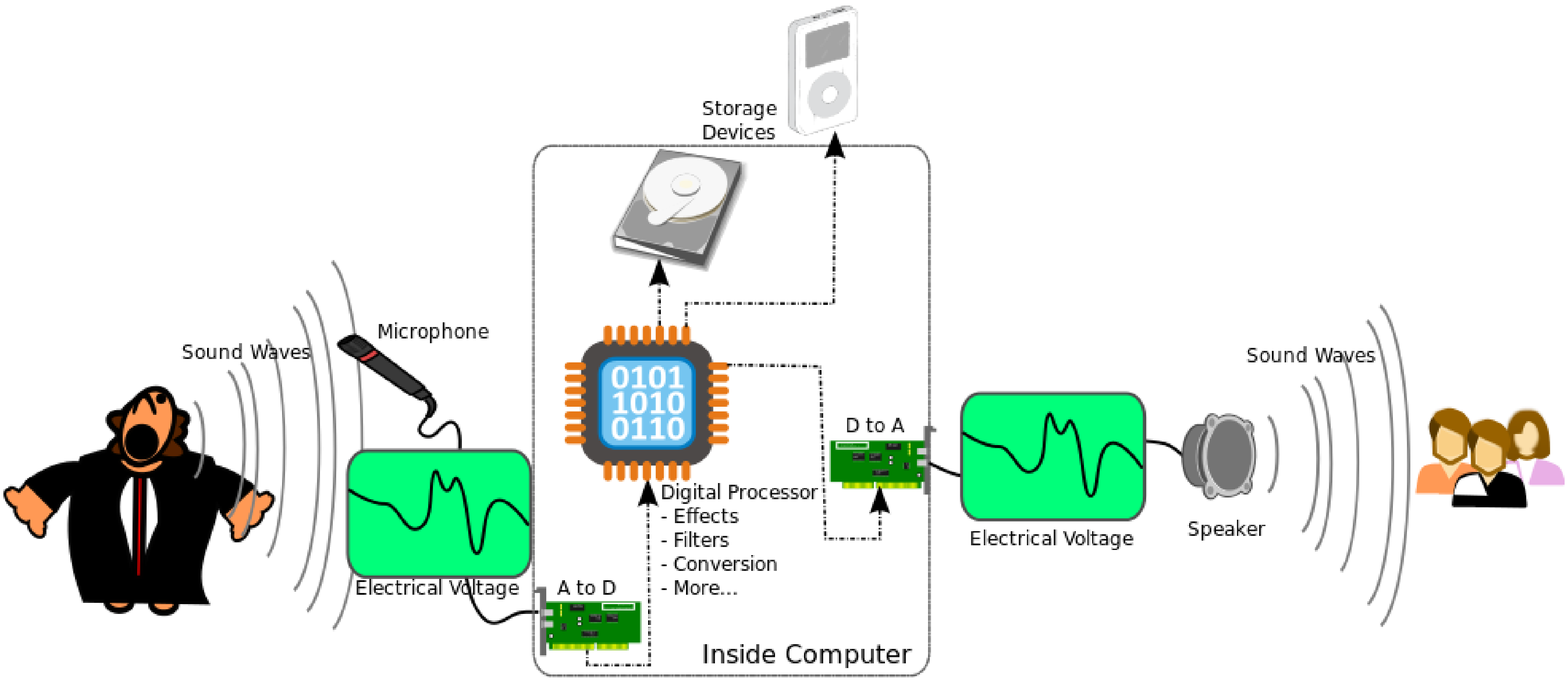
spherical pressure waves

Digital Sound

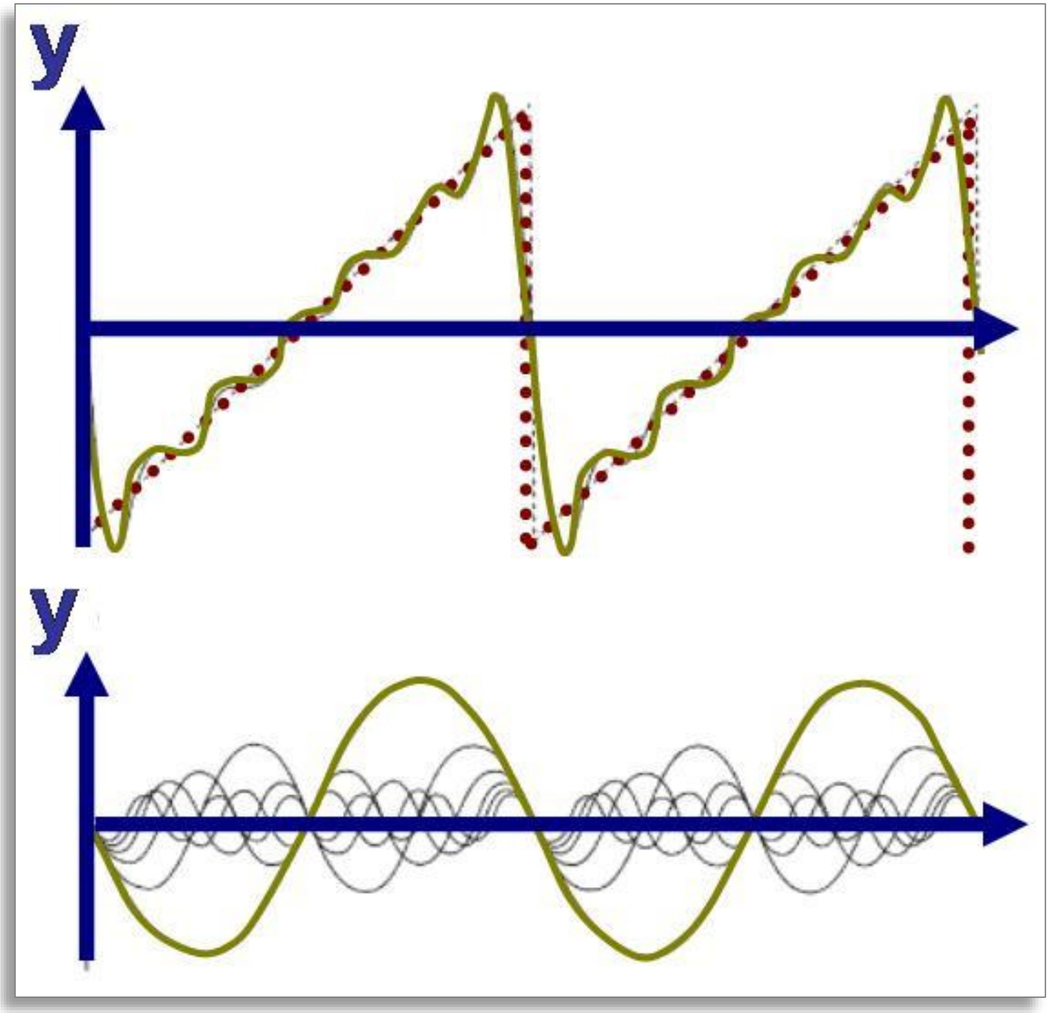


Digitizing / sampling an analog sound signal

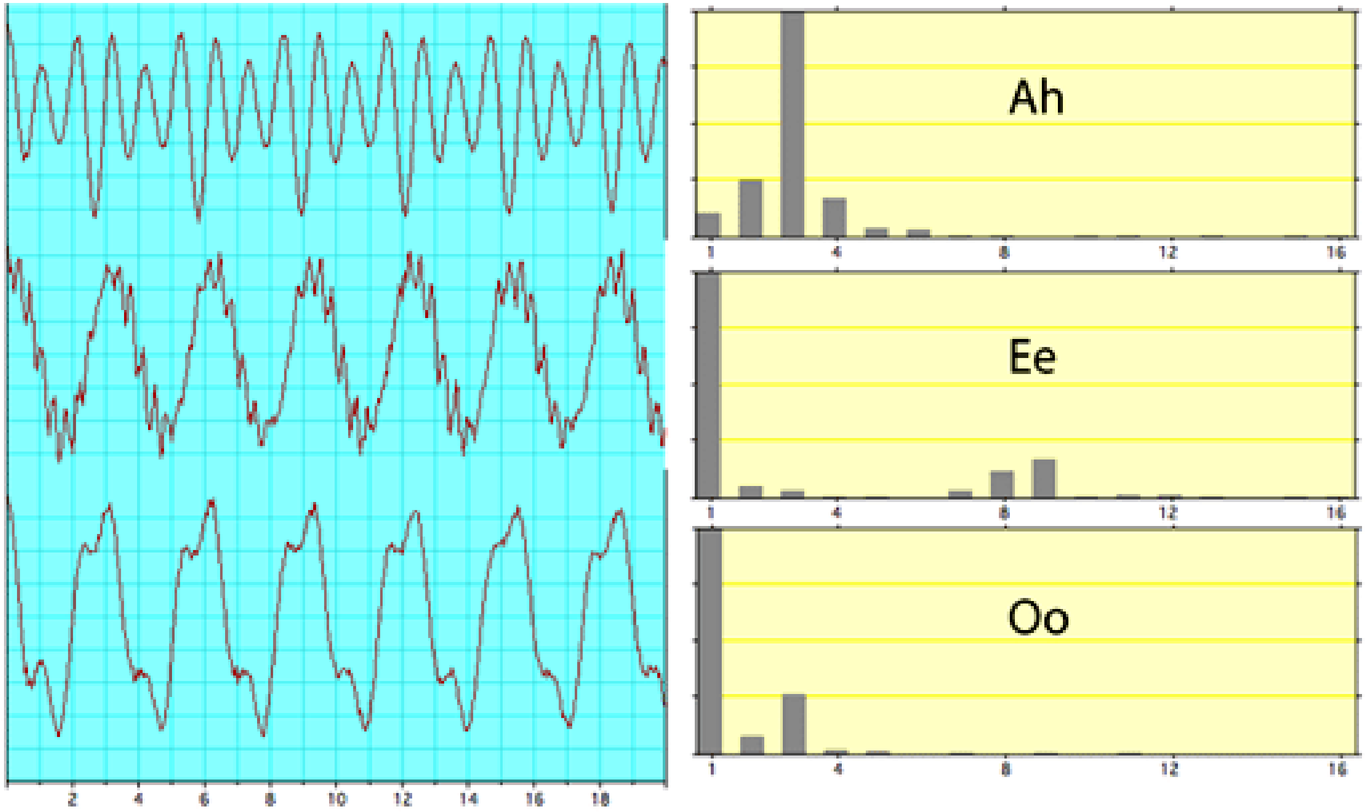
Digital Sound Processing



Fourier Analysis



Analysing Sound in the Frequency Domain



Functional Sound Synthesis

- **Simple example:**

Show in Playground

```
const pitch_A_wave =
    t => math_sin(2 * math_PI * 440 * t); // A4

const pitch_A = make_sound(pitch_A_wave, 1.5);

play(pitch_A);
```

Functional Sound Synthesis

- **Second example:**

Show in Playground

```
const C_maj_chord_wave =
    t => 0.33 * math_sin(2 * math_PI * 261.63 * t) + // C4
         0.33 * math_sin(2 * math_PI * 329.63 * t) + // E4
         0.33 * math_sin(2 * math_PI * 392.00 * t);  // G4

const C_maj_chord = make_sound(C_maj_chord_wave, 1.5);

play(C_maj_chord);
```

Functional Sound Synthesis

- **Third example:**

Show in Playground

```
const doremi_wave =
    t => t < 0.5
         ? math_sin(2 * math_PI * 261.63 * t)  // C4
         : t < 1.0
         ? math_sin(2 * math_PI * 293.66 * t)  // D4
         : math_sin(2 * math_PI * 329.63 * t); // E4

const doremi = make_sound(doremi_wave, 1.5);

play(doremi);
```

Sound in Source

- Supported by the **sound module**

- Uses **functional sound**

Sound from Past Contests

Summary

- Sound is a physical phenomenon
- We digitize sound to process it in the computer
- To hear digitized sound, we need to “undigitize“ it
- In CS1101S, we do it differently: we work with functional sound